

Summary of Kareus: Joint Reduction of Dynamic and Static Energy in Large Model Training

Vansh Nagpal (vnagpal), Anshul Batish (abatish), Shrey Kohli (kohlshr), Pranav Gupta (gpranav)

Problem and Motivation

This paper studies how to reduce the energy cost of training large AI models while still maintaining strong performance. As model sizes and training runs continue to grow, energy is becoming a major systems bottleneck. Training large language models now consumes enormous amounts of electricity, which increases financial cost and raises concerns about scalability and sustainability.

The authors argue that existing approaches usually optimize only one part of the energy problem. Some methods reduce dynamic energy, which is the energy consumed when the GPU is actively computing, usually by lowering GPU frequency (through a process known as Dynamic Voltage and Frequency Scaling, aka DVFS). Other methods reduce static energy (due to leakage voltage), which is the energy wasted simply because the GPU remains powered on for longer due to delays, communication overhead, or poor overlap between tasks. The key problem is that these two forms of energy are not independent. A training schedule that seems good for one may be bad for the other.

This problem matters because modern large-model training involves heavy communication across GPUs due to tensor parallelism, pipeline parallelism, and other distributed methods. If communication is not handled efficiently, GPUs spend more time waiting, which wastes energy. At the same time, simply throttling the GPU to save power may increase total training time and cancel the benefit. Therefore, reducing training energy requires a joint view of computation, communication, and hardware settings.

Related Works

The paper builds on two broad areas of prior work.

The first area focuses on GPU power and frequency management for machine learning workloads. Systems such as Zeus and Perseus showed that GPU frequency scaling can lower energy use without necessarily hurting performance. These methods are important because they demonstrate that training does not always need to run at maximum clock speed. However, they mainly target dynamic power consumption and do not deeply optimize the training execution schedule itself.

The second area focuses on communication-computation overlap. Existing works attempt to reduce overhead by overlapping communication kernels with useful computation. These methods improve utilization and often reduce total iteration time. Since faster execution also reduces the time GPUs stay active, they help cut static energy indirectly.

However, the paper argues that prior work remains incomplete because these two directions are usually treated separately. Frequency scaling is optimized on one side, and overlap scheduling on the other. In reality, the best GPU frequency depends on how communication and computation are overlapped, and the best overlap strategy depends on the chosen frequency and resource allocation. This gap in prior work motivates Kareus.

Solution Overview

The authors of Kareus propose jointly optimizing energy and performance during large-model training. The main idea is that energy efficiency should be treated as a joint scheduling problem, not as separate tuning of clock speed or overlap. Kareus determines how execution schedules impact time and energy consumption, and that execution schedules are a combination of: GPU frequency, SM allocation for communication kernels, and Kernel launch timing to determine how communication overlaps with computation

The authors' central insight is that the execution schedule affects both static and dynamic energy simultaneously. Better overlap shortens execution and reduces static energy. Better frequency settings reduce dynamic power. But these interact: lower frequency changes kernel behavior, and overlap choices can create contention if launched poorly. Therefore, the system must optimize them together rather than one after another.

To make this feasible, Kareus breaks the training iteration into smaller repeating partitions based on common Transformer communication-computation patterns. Instead of searching the full schedule space at once, it profiles and optimizes these local partitions. For each partition, it uses multi-objective Bayesian optimization to search for good tradeoffs between time and energy. It then combines these local Pareto-optimal schedules into an efficient end-to-end execution plan.

This design is important because the search space is very large, making brute-force exploration impractical. By structuring the problem around repeated patterns and using Bayesian optimization, Kareus can find strong schedules without exhaustively profiling everything.

The results show that Kareus improves the time-energy tradeoff frontier compared to prior baselines. According to the paper, it can reduce energy by up to 28.3% at the same training time, or reduce training time by up to 27.5% under the same energy budget. This shows that joint optimization is significantly better than treating overlap and frequency scaling independently.

Limitations

One limitation of Kareus is that it depends on careful profiling and optimization overhead. Even though it avoids exhaustive search, it still needs enough profiling data to build good partition-level tradeoff frontiers. This makes deployment harder in settings where workloads change frequently or where there is little time for tuning.

Future Research Directions

One direction is to extend Kareus from a single training job to cluster-level energy scheduling. Instead of optimizing only one workload, future systems could use similar ideas to allocate power budgets across many jobs in a datacenter.

Summary of Class Discussion

- ❖ Does combining partition-wise results risk sacrificing global optimality because each partition has its own bottlenecks?
 - Ans: Possibly, but GPU throttling and switching overhead are more harmful in practice.
- ❖ If random initialization already finds 35% of the final frontier candidates, do we really need such a complex method?
 - Ans: Yes, because the optimization passes must work together, and MBO is far more efficient than exhaustive search.
- ❖ Does the paper guarantee that combining independently optimized partition frontiers gives a globally optimal result?
 - Ans: Not full global optimality, but it guarantees suboptimal candidates are filtered out early.
- ❖ What is static energy and dynamic energy?
 - Ans: Dynamic energy is the energy used for actual computation; static energy is the idle energy consumed just by keeping the GPU on.
- ❖ Can Kareus be extended to inference?
 - Ans: It is harder for inference because batch sizes and token counts vary, so each shape may need its own optimized configuration.
- ❖ How difficult is it to deploy Kareus on systems with different schedulers?
 - Ans: It likely would not be very difficult and should not require major changes.
- ❖ Since the Kareus optimizer is greedy, can it miss better interactions?
 - Ans: Yes, potentially, but the exploration and additional passes are meant to reduce that risk.

Summary of "TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms"

Problem and Motivation

This paper studies how cloud providers can manage the thermal and power challenges of LLM inference. LLM serving runs on very power-hungry GPUs, and inference behavior changes across phases such as prefill and decode. In addition, operators can tune model size, quantization, tensor parallelism, GPU frequency, and batch size, all of which affect performance, temperature, power, and sometimes output quality. Because of this, traditional datacenter power and cooling policies are no longer sufficient.

GPU clusters are increasingly limited by power density and cooling capacity. If thermal and power hotspots are not controlled, GPUs can throttle, rows can exceed power budgets, and datacenter capacity gets wasted. The paper argues that better scheduling can reduce throttling and safely increase oversubscription, lowering the total cost of ownership.

Related Works

The paper builds on three areas of prior work. First, there is extensive work on LLM inference performance, including serving systems and optimizations for throughput and latency. However, these systems usually do not focus on datacenter-level thermal and power constraints.

Second, prior work on datacenter thermal and power management has studied cooling efficiency, power capping, and infrastructure failures. But TAPAS argues that LLM inference behaves differently from traditional workloads because it has millisecond-scale phase changes and several workload knobs that affect both infrastructure stress and model quality.

Third, cloud systems already use VM placement and load balancing, but these are often thermal- and power-oblivious. They may distribute the load evenly for performance while accidentally worsening hotspots. TAPAS extends these ideas by making them infrastructure-aware.

Solution Overview

The authors propose TAPAS, a thermal and power-aware scheduling framework for LLM inference clusters in the cloud. Its key insight is that cloud providers should use the predictability and flexibility of SaaS workloads to shape thermal and power demand across the datacenter, while minimizing impact on opaque IaaS workloads.

TAPAS works in three ways.

First, it performs thermal- and power-aware VM placement. It uses historical data to predict airflow demand, row power, and likely GPU temperatures, then avoids placements that would create aisle airflow violations, row power violations, or overheated servers. IaaS VMs are preferably placed on cooler servers because they are harder to control, while SaaS VMs can be placed more flexibly since they can later be adapted.

Second, TAPAS performs request routing across SaaS instances. It checks aisle, row, and server conditions before routing requests, and avoids sending traffic to VMs that could trigger thermal or power problems. This allows the load balancer to smooth infrastructure stress instead of amplifying hotspots.

Third, TAPAS performs instance reconfiguration. When load spikes or failures occur, it changes SaaS instance settings such as GPU frequency, batch size, tensor parallelism, quantization, or model size to keep the system within thermal and power limits while still satisfying performance SLOs. It chooses the highest-accuracy, highest-performance configuration that remains safe.

The paper shows that TAPAS maintains P99 latency while reducing maximum temperature by 17% and peak row power by 23%, enabling up to 40% extra capacity through safer oversubscription. At a larger scale, it reduces thermal throttling events by 97% and power throttling events by 99%.

Limitations

TAPAS has a few important limitations. First, it depends strongly on historical data and prediction accuracy. Its placement, routing, and control decisions rely on estimating VM load, row power, and temperature behavior from past observations. This works well for long-lived workloads with regular diurnal patterns, but may be less reliable when workloads shift suddenly or become less predictable. If these estimates are wrong, TAPAS may make weaker scheduling decisions. In addition, the system works best when there is a large amount of flexible SaaS workload, because SaaS instances can be routed and reconfigured, while IaaS workloads remain mostly opaque and cannot be controlled in the same way. As a result, clusters dominated by IaaS or with only a small SaaS presence would likely see smaller benefits.

A second limitation is practical deployment overhead. Some of the configuration knobs TAPAS uses, such as changing model parallelism, model size, or quantization, may require restarting and reloading the model, which is more disruptive than lightweight controls like routing or frequency scaling. This means not all optimization actions are equally easy to apply in practice. Also, the paper evaluates TAPAS in Microsoft Azure datacenter environments and uses regression models tailored to those settings. Although the core ideas are likely more general, deploying TAPAS elsewhere would still require retraining or recalibrating the models for different hardware, cooling systems, and datacenter layouts.

Future Research Directions

TAPAS can be extended to other GPU-heavy AI services, such as diffusion or multimodal serving. The general idea should transfer, but the profiling and control knobs would need to be adjusted. Another direction is supporting live GPU VM migration, especially for IaaS workloads. This would allow the system to rebalance the datacenter more dynamically instead of depending mostly on initial placement. The framework can also be adapted to different cooling and power infrastructures, such as liquid cooling or different datacenter power hierarchies, to test how portable the method is.

Summary of Class Discussion

- ❖ Is there a concern that fixing one hotspot could just create another hotspot somewhere else?
 - Ans: The paper does not deeply analyze cascading hotspots, but TAPAS tries to avoid this by balancing workloads across rows and other infrastructure levels.
- ❖ TAPAS often looks at a roughly 50–50 split between IaaS and SaaS. What happens with different split ratios?
 - Ans: The authors do study different splits; TAPAS works better when there is more SaaS because SaaS is much more flexible than IaaS.
- ❖ If, in the future, much smaller models achieve almost no performance loss, how would this affect TAPAS?
 - Ans: TAPAS already uses smaller models when helpful, but if models become very small and cheap, some inference may move outside large datacenters altogether.
- ❖ TAPAS assumes SaaS workloads can be reconfigured. How does it manage quality expectations?
 - Ans: It treats model downgrades as a last resort, mainly during emergencies, so quality is usually kept stable and only drops rarely.
- ❖ What does it mean that 65% of candidates are found? Are the other 35% simply missed?
 - Ans: No. The remaining 35% are mostly rejected as suboptimal, and the optimization is done per partition rather than missing useful candidates globally.